WYDZIAŁ
ELEKTROTECHNIKI
I INFORMATYKI
POLITECHNIKI RZESZOWSKIEJ

Anna Pitera

Joanna Lichtarska
Arkadiusz Połeć

Raport bezpieczeństwa dla aplikacji webowej

Rzeszów, 2024

Spis treści

1. Wstęp **4**

2. Faza planowania projektu **5**

2.1. Przegląd standardów - technologie 5

2.2. Historia problemów bezpieczeństwa wybranych technologii 8

2.2.1. Zalecenia, standardy i najlepsze praktyki bezpieczeństwa w Django 13

2.2.2. Zalecenia, standardy i najlepsze praktyki bezpieczeństwa w React 15

2.2.3. Zalecenia, standardy i najlepsze praktyki bezpieczeństwa dla bazy PostgreSQL 16

3. Faza projektowania rozwiązań **18**

3.1. Przegląd wymagań bezpieczeństwa 18

3.2. Weryfikacja architektury projektu 27

4. Faza tworzenia aplikacji **30**

4.1. Implementacja backendu i frontendu 30

4.2. Projektowanie interfejsu użytkownika 35

4.3. Przegląd przepływu danych w aplikacji 40

4.4. Kontrola kodu i zapewnienie bezpieczeństwa 40

5. Faza wdrażania aplikacji - testy aplikacji **44**

5.1. Proces wdrażania aplikacji 44

5.2. Testy aplikacji 46

6. Wnioski **50**

7. Literatura **51**

1. Wstęp

Celem projektu jest stworzenie aplikacji webowej do nauki słówek, która pozwoli użytkownikom na dodawanie nowych słów, definiowanie ich oraz przeprowadzanie testów wiedzy. Aplikacja będzie składać się z backendu napisanego w Django oraz frontendu opartego na React. Komunikacja między frontendem a backendem będzie odbywać się za pomocą API REST.

Aplikacja składa się z trzech głównych komponentów:

- **Backend** zbudowany z użyciem frameworku Django, zarządza logiką aplikacji oraz komunikacją z bazą danych. Wykorzystuje relacyjną bazę danych do przechowywania informacji o użytkownikach oraz ich postępach w nauce słówek. Zapewni mechanizm uwierzytelniania i autoryzacji użytkowników za pomocą JSON Web Tokens (JWT)

- **Frontend** stworzony w technologii React, korzystający z HTTP do komunikacji z backendem za pomocą REST API.

- **Baza danych** stworzona w PostgreSQL będzie zawierać co najmniej trzy tabele związane z funkcjonalnością aplikacji oraz tabelę użytkowników.

2. Faza planowania projektu

2.1. Przegląd standardów - technologie

Podczas tworzenia aplikacji webowej do nauki słówek, zdecydowano się na Django jako framework backendowy oraz React jako bibliotekę frontendową. Jako system zarządania obiektowo-relacyjną bazą danych wybrano PostgreSQL. Istnieją jednak inne technologie, które mogłyby zostać użyte do budowy takiej aplikacji, każda z nich mając swoje zalety i wady, a także specyficzne standardy bezpieczeństwa. Poniżej przedstawiono przegląd niektórych alternatywnych technologii oraz ich standardy z uzględnieniem uzasadnienia finalnego wyboru.

Backend

1) **Django (Python)** - framework typu "batteries-included", co oznacza, że oferuje gotowe komponenty do wielu typowych zadań, takich jak uwierzytelnianie użytkowników, zarządzanie bazami danych, szablony HTML, mechanizmy sesji, oraz zarządzanie statycznymi plikami. To pozwala na szybkie budowanie aplikacji webowych bez konieczności pisania kodu od zera dla tych funkcji. Niezwykle istotne jest to, że ma on wbudowane mechanizmy zabezpieczające przed wieloma typowymi atakami webowymi, takimi jak SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), Clickjacking. Framework automatycznie stosuje odpowiednie techniki, aby minimalizować ryzyko tych zagrożeń, co sprawia, że aplikacje są bezpieczniejsze od samego początku. Django jest bardziej monolityczny w porównaniu do innych mikroframeworków, takich jak Flask. Oznacza to, że ma więcej wbudowanych komponentów, co może być zarówno zaletą, jak i wadą, w zależności od wymagań projektu. [1]

2) **Flask (Python)** - mikroframework dla Pythona, który oferuje większą elastyczność niż Django. Jednak oferuje minimalny zestaw funkcji, co oznacza, że wiele funkcjonalności, takich jak uwierzytelnianie czy migracje baz danych, musi być dodawane ręcznie za pomocą zewnętrznych bibliotek. Dodatkowo Flask może być mniej skalowalny w porównaniu do bardziej rozbudowanych frameworków, takich jak Django, ze względu na brak wbudowanych narzędzi do zarządzania dużymi aplikacjami. [2]

3) **Express.js (Node.js)** - minimalistyczny framework webowy dla Node.js, który oferuje dużą elastyczność i wydajność. Jednak duża elastyczność może prowadzić do braku spójności w kodzie w większych zespołach deweloperskich. Ponadto wiele funkcjonalności musi być zarządzane ręcznie, co może zwiększać złożoność kodu i ryzyko błędów. [3]

4) **Ruby on Rails (Ruby)** - framework webowy dla języka Ruby, który oferuje bogaty zestaw funkcji out-of-the-box. Promuje on filozofię "konwencja nad konfiguracją", co oznacza, że domyślne ustawienia i konwencje frameworku zmniejszają ilość konfiguracji potrzebnej do uruchomienia aplikacji. Jednak Rails jest stosunkowo zasobochłonny, co może wpływać na koszty hostingowe, a ze względu na liczbę wbudowanych funkcji i konwencji, może być trudniejszy do nauczenia się dla początkujących. [4]

Frontend

1) **React (JavaScript)** - biblioteka języka programowania JavaScript, która używa wirtualnego DOM, co poprawia wydajność aplikacji. Dodatkowo React promuje budowanie aplikacji złożonych z małych, niezależnych komponentów, które można łatwo zarządzać, ponownie wykorzystywać i testować. Komponentowa architektura ułatwia utrzymanie i rozwijanie kodu. Niestety React nie oferuje wielu funkcji out-of-the-box, co może wymagać dodatkowej konfiguracji, a jego ekosystem zmienia się szybko, co może wymagać ciągłego uczenia się nowych narzędzi i bibliotek. [5]

2) **Angular (TypeScript)** - framework do budowy aplikacji webowych, który oferuje wiele wbudowanych funkcji takich jak routing, formy, walidacje. Wykorzystuje TypeScript, co pozwala na statyczne typowanie i lepsze zarządzanie dużymi kodami. Aczkolwiek Angular jest złożonym frameworkiem, co może sprawić, że jest trudny do nauki dla początkujących, a ze względu na swoją złożoność, Angular może być mniej wydajny w porównaniu do lżejszych frameworków. [6]

3) **Vue.js (JavaScript)** - progresywny framework do budowy interfejsów użytkownika, który jest łatwy w integracji z innymi projektami. Jednakże ma mniejszą

społeczność w porównaniu do Angulara i Reacta, co może oznaczać mniej dostępnych zasobów i wsparcia. [6]

4) **Svelte (JavaScript)** - framework do budowy interfejsów użytkownika, który różni się od innych tym, że wykonuje większość swojej pracy kompilacji w czasie budowania, a nie w przeglądarce. Niestety ekosystem Svelte jest mniej dojrzały, co oznacza mniej dostępnych wtyczek i narzędzi, dodatkowo ma mniejszą społeczność w porównaniu do innych frameworków, co może ograniczać dostępność zasobów i wsparcia. [7]

Bazy danych

1) **PostgreSQL** - baza danych obsługująca wiele typów danych, w tym JSON, XML, i inne niestandardowe typy. PostgreSQL oferuje zaawansowane mechanizmy kontroli dostępu i uwierzytelniania, a także wiele zaawansowanych funkcji, takich jak transakcje ACID, widoki materializowane, procedury składowane i replikacja. Choć PostgreSQL jest skalowalny, jego skalowanie poziome może być bardziej skomplikowane w porównaniu do baz danych NoSQL, takich jak MongoDB. Dodatkowo ma bardziej złożoną konfigurację i zarządzanie w porównaniu do prostszych systemów baz danych, co może wymagać większej wiedzy i doświadczenia. [8]

2) **MySQL** - popularna baza danych typu open source, szeroko stosowana w aplikacjach webowych. Oferuje dobrą wydajność i jest zoptymalizowany do obsługi dużej liczby zapytań. Posiada wiele funkcji bezpieczeństwa, takich jak uwierzytelnianie i kontrola dostępu. MySQL może mieć jednak ograniczenia w zakresie skalowalności w porównaniu do niektórych systemów NoSQL. [8]

3) **MongoDB** - baza danych NoSQL, która przechowuje dane w formie dokumentów JSON, co pozwala na łatwe przechowywanie złożonych struktur danych. Oferuje wysoką wydajność, szczególnie w przypadku aplikacji wymagających szybkiego dostępu do danych, a także jest dobrze przystosowany do skalowania poziomego, co umożliwia łatwe zarządzanie dużymi ilościami danych. Jednak do mniejszych projektów jego zastosowanie mogłoby być lekkim przerostem formy nad treścią. [8]

Po przeprowadzonej analizie zalet w zakresie funkcjonalności, bezpieczeństwa, wydajności oraz wsparcia społeczności potwierdzono słuszność wyboru Django jako frameworku backendowego, React jako frameworku frontendowego oraz PostgreSQL jako bazy danych. Każda z tych technologii przyczynia się do stworzenia nowoczesnej, bezpiecznej i wydajnej aplikacji webowej, która spełnia współczesne wymagania i standardy oraz mieści się w naszym zakresie umiejętności. Oczywiście żadna z nich nie jest wolna od wad, jednak jesteśmy ich świadomi.

2.2. Historia problemów bezpieczeństwa wybranych technologii

Przypadki znanych luk bezpieczeństwa w Django:

1) **CSRF (Cross-Site Request Forgery) w wersjach przed 1.2.**

   CSRF to atak polegający na wykonaniu nieautoryzowanych akcji na stronie internetowej przez użytkownika, który został sfałszowany przez atakującego. Przed wersją 1.2 Django nie zawierał domyślnie mechanizmu ochrony przed CSRF, co prowadziło do podatności na tego rodzaju ataki.

   Wersja 1.2 Django wprowadziła mechanizm zabezpieczający przed CSRF, który polega na wygenerowaniu unikalnego tokena dla każdego formularza i sprawdzeniu go podczas przetwarzania żądania. Ten mechanizm zapobiega atakom CSRF poprzez wymuszenie, aby żądanie zawierało poprawny token CSRF. Wymaga to użycia tagu szablonu csrf_token w szablonie. CsrfViewMiddleware jest domyślnie dołączony do MIDDLEWARE_CLASSES. Domyślnie włącza to ochronę CSRF, więc widoki akceptujące żądania POST muszą być napisane tak, aby działały z oprogramowaniem pośredniczącym. [9]

2) **SQL Injection w wersjach przed 1.2.5.**

   SQL Injection to atak polegający na wstrzyknięciu złośliwego kodu SQL do zapytania SQL wykonywanego przez aplikację. Przed wersją 1.2.5 Django nie zapewniał wystarczających mechanizmów zabezpieczających przed tego rodzaju atakami, co mogło prowadzić do wycieku danych lub uszkodzenia bazy danych.

   Wersja 1.2.5 Django wprowadziła szereg poprawek, które zwiększyły bezpieczeństwo aplikacji poprzez lepsze zabezpieczenia przed atakami SQL Injection. Django zachęca do korzystania z mechanizmów ORM (Object-Relational

Mapping) oraz parametryzacji zapytań SQL w celu zapobiegania tego rodzaju atakom. [9]

3) **XSS (Cross-Site Scripting) w wersjach przed 1.2.7.**

XSS to atak polegający na wstrzyknięciu złośliwego kodu JavaScript do strony internetowej, który jest następnie wykonany przez przeglądarkę użytkownika. Przed wersją 1.2.7 Django nie dostarczał wystarczających mechanizmów zabezpieczających przed XSS, co mogło prowadzić do wykonywania złośliwego kodu w kontekście aplikacji.

Wersja 1.2.7 Django wprowadziła różne poprawki zabezpieczeń, które zwiększyły odporność aplikacji na ataki XSS. Django zaleca ostrożne używanie funkcji renderowania zawartości, takich jak dangerouslySetInnerHTML, oraz automatyczne eskapowanie danych w celu zapobieżenia XSS. [9]

4) **Host header poisoning w wersjach 1.3.3 i 1.4.1**

Parsowanie nagłówka Host w Django 1.3.3 i Django 1.4.1, a konkretnie django.http.HttpRequest.get_host() - nieprawidłowo obsługiwało informacje o nazwie użytkownika/hasła w nagłówku. W związku z tym, np. następujący nagłówek "Host" zostałby zaakceptowany przez Django podczas działania na validsite.com: "Host: validsite.com:random@evilsite.com"

Wykorzystując to, atakujący może spowodować, że niektóre części Django (w szczególności mechanizm resetowania hasła) generują i wyświetlają użytkownikom arbitralne adresy URL.

Aby zaradzić temu problemowi, parsowanie w HttpRequest.get_host() jest modyfikowane. Nagłówki "Host", które zawierają potencjalnie niebezpieczne treści (takie jak pary nazwa użytkownika/hasło), teraz powodują zgłoszenie wyjątku django.core.exceptions.SuspiciousOperation. [9]

5) **Denial-of-service za pomocą hasherów haseł w wersjach przed 1.4.8.**

W poprzednich wersjach Django nie było nałożonego limitu na długość hasła w postaci zwykłego tekstu. Pozwalało to na atak typu denial-of-service poprzez przesyłanie fałszywych, ale ekstremalnie dużych haseł, wiążąc zasoby serwera wykonujące (kosztowne i coraz droższe wraz z długością hasła) oblicza-

nie odpowiedniego hasha.

Od wersji 1.4.8, framework uwierzytelniania Django nakłada 4096-bajtowy limit na hasła i nie powiedzie się uwierzytelnienie z każdym przesłanym hasłem o większej długości. [9]

6) **Nieoczekiwane wykonanie kodu za pomocą reverse() w wersjach przed 1.4.11**

Django zarządza adresami URL aplikacji, mapując wzorce regex (regular expressions) do odpowiednich widoków. Kiedy użytkownik odwiedza stronę internetową, Django używa tych wzorców, aby zdecydować, który widok wywołać w odpowiedzi na konkretne żądanie URL.

Funkcja reverse() działa w odwrotny sposób niż zwykłe mapowanie adresów URL. Zamiast mapować wzorzec URL do widoku, reverse() bierze informacje o widoku (np. nazwę widoku) i zwraca odpowiadający mu adres URL.

Jednym z argumentów dla reverse() jest kropkowana ścieżka Pythona do widoku. Gdy Django używa tej funkcji, importuje moduł wskazany przez tę ścieżkę, aby wygenerować adres URL. Jeśli moduł ma jakieś efekty uboczne podczas importowania (np. wykonuje jakiś kod), te efekty uboczne zostaną uruchomione.

Atakujący może spowodować nieoczekiwane wykonanie kodu, jeśli spełnione są dwa warunki:

(a) Istnieją widoki, które konstruują adres URL na podstawie wejścia użytkownika (np. parametr "next"w zapytaniu, który mówi, gdzie przekierować użytkownika po zakończeniu pewnej akcji).

(b) Atakujący zna moduły na ścieżce importu Pythona serwera, które wykonują kod z efektami ubocznymi podczas importu.

W takim przypadku atakujący mógłby wstrzyknąć złośliwą kropkowaną ścieżkę Pythona w parametr "next", powodując importowanie modułu, który wykonuje niechciany kod.

Aby temu zaradzić, Django wprowadziło ograniczenie w funkcji reverse(). Teraz reverse() akceptuje i importuje tylko kropkowane ścieżki modułów, które

są wymienione w konfiguracji wzorców URL projektu. Oznacza to, że tylko moduły, które są wyraźnie dozwolone przez programistę, mogą być importowane w ten sposób, co znacznie zmniejsza ryzyko nieoczekiwanego wykonania kodu. [9]

7) **Typcasting MySQL w wersjach przed 1.4.11**

Baza danych MySQL może zmieniać typy w zapytaniach, co może prowadzić do nieoczekiwanych wyników. Klasy pól modelowych Django wykonują wyraźną konwersję typów argumentów zapytania przed ich wykonaniem, ale trzy klasy pól modelowych (FilePathField, GenericIPAddressField, IPAddressField) tego nie robiły. Zostały one zaktualizowane, aby konwertować swoje argumenty przed zapytaniem. Programiści niestandardowych pól modelowych i użytkownicy metod raw() oraz extra() są teraz ostrzegani, aby zapewnili odpowiednie konwersje typów przed wykonaniem zapytań. [9]

Przypadki znanych luk bezpieczeństwa w React:

1) **XSS (Cross-Site Scripting) w wersjach przed 15.0.**

Cross-Site Scripting (XSS) to atak, który polega na wstrzyknięciu złośliwego kodu JavaScript do strony internetowej. W kontekście React, XSS może wystąpić, gdy aplikacja renderuje dane pochodzące od użytkownika bez odpowiedniej walidacji i eskapowania.

W starszych wersjach React (przed 15.0) istniały luki umożliwiające ataki XSS. Przykładowo, bezpośrednie używanie 'dangerouslySetInnerHTML' bez odpowiedniego oczyszczenia danych mogło pozwolić na wstrzyknięcie złośliwego kodu JavaScript do komponentów.

React domyślnie eskapuje dane wstawiane do DOM, co znacznie utrudnia przeprowadzenie ataków XSS. Dodano również możliwość korzystania z dangerouslySetInnerHTML, ale z ostrzeżeniem o ryzyku.

Domyślne eskapowanie (czyli zamiana znaków specjalnych, które mogłyby być wykorzystane do wstrzyknięcia złośliwego kodu JavaScript, na bezpieczne odpowiedniki) w React skutecznie chroni przed XSS, pod warunkiem że dangerouslySetInnerHTML jest używane tylko w uzasadnionych przypadkach z odpowiednimi zabezpieczeniami.

W praktyce oznacza to, że gdy dane są wstawiane do elementów DOM w React, framework automatycznie zamienia znaki takie jak:

```
"<", ">", "&"
```

aby były one renderowane jako tekst, a nie jako część kodu HTML lub JavaScript. Na przykład, jeśli dane zawierają znaki:

```
<script>alert('złośliwy kod');</script>,
```

React automatycznie zamieni te znaki na ich encje HTML, co uniemożliwi interpretację tego jako kodu JavaScript przez przeglądarkę.

Dzięki domyślnemu eskeplowaniu w React, programiści nie muszą ręcznie manipulować danymi w celu zapobieżenia XSS, co znacznie ułatwia tworzenie bezpiecznych aplikacji webowych. [5]

2) **Problemy z zarządzaniem stanem komponentów.**

Zarządzanie stanem w React jest kluczowe dla utrzymania spójności i przewidywalności aplikacji. Problemy mogą pojawić się, gdy stan jest zarządzany w sposób nieoptymalny lub niespójny, co może prowadzić do błędów lub luk bezpieczeństwa.

Starsze wersje React miały problemy z zarządzaniem stanem komponentów, co mogło prowadzić do nieprzewidzianych błędów w aplikacji.

Nowe wersje React wprowadziły zaawansowane narzędzia do zarządzania stanem, takie jak Hooki (useState, useEffect) i Context API. Poprawiło to stabilność aplikacji i zmniejszyło liczbę błędów związanych z niepoprawnym zarządzaniem stanem. [5]

3) **Podatności związane z deserializacją**

Problem deserializacji danych był obecny w różnych wersjach zależnych bibliotek używanych przez React. W niektórych przypadkach deserializacja danych pochodzących od użytkownika mogła prowadzić do wykonania złośliwego kodu, jeśli dane nie były odpowiednio walidowane. Zwiększono więc nacisk na walidację danych pochodzących od użytkownika i zalecenie unikania bezpośredniej deserializacji danych niezweryfikowanych. [5]

4) **Zewnętrzne biblioteki i komponenty**

React jest często używany w połączeniu z wieloma zewnętrznymi bibliotekami i komponentami, które mogą zawierać własne luki bezpieczeństwa. Wprowadzono więc regularne audyty zależności oraz korzystanie z narzędzi takich jak npm audit, aby monitorować i aktualizować podatne zależności. [5]

2.2.1. Zalecenia, standardy i najlepsze praktyki bezpieczeństwa w Django

Django jest jednym z najpopularniejszych frameworków do tworzenia aplikacji webowych w Pythonie, który został zaprojektowany z myślą o bezpieczeństwie. Framework oferuje wiele wbudowanych mechanizmów, które pomagają deweloperom w tworzeniu bezpiecznych aplikacji. Poniżej przedstawiono kluczowe zalecenia, standardy i najlepsze praktyki dotyczące bezpieczeństwa w Django.

- **Ochrona przed atakami typu CSRF (Cross-Site Request Forgery)**

  Ataki CSRF umożliwiają złośliwemu użytkownikowi wykonywanie działań przy użyciu poświadczeń innego użytkownika bez jego wiedzy. Django posiada wbudowaną ochronę przed większością ataków CSRF, która polega na sprawdzaniu sekretu w każdym żądaniu POST. Należy unikać wyłączania tej ochrony bez odpowiedniej wiedzy. Zawsze należy używać HTTPS, aby zwiększyć skuteczność ochrony CSRF oraz unikać używania csrf_exempt, chyba że jest to absolutnie konieczne.

- **Ochrona przed atakami typu XSS (Cross-Site Scripting)**

  Ataki XSS pozwalają użytkownikowi na wstrzyknięcie skryptów po stronie klienta do przeglądarek innych użytkowników. Django chroni przed większością ataków XSS poprzez automatyczne kodowanie niebezpiecznych znaków w szablonach HTML, jednak nie jest to ochrona absolutna. Należy unikać wyłączania automatycznego kodowania oraz ostrożnie używać funkcji takich jak mark_safe.

- **Ochrona przed atakami typu SQL Injection**

  Ataki SQL Injection pozwalają na wykonanie dowolnego kodu SQL na bazie danych. Django chroni przed tymi atakami poprzez parametryzację zapytań w QuerySets. Należy unikać bezpośredniego wykonywania surowych zapytań SQL lub dokładnie oczyszczać wszelkie parametry wprowadzane przez użytkownika.

- **Ochrona przed Clickjackingiem**

  Clickjacking to atak, w którym złośliwa strona osadza inną stronę w ramce. Django zapewnia ochronę przed tymi atakami za pomocą middleware X-Frame-Options, które zapobiega renderowaniu strony w ramkach przez zewnętrzne witryny.

- **SSL/HTTPS**

  Zawsze warto wdrożyć aplikację za pomocą HTTPS, aby zapobiec podsłuchiwaniu i manipulowaniu danymi przesyłanymi między klientem a serwerem. Należy ustawić SECURE_SSL_REDIRECT na True oraz używać bezpiecznych ciasteczek (SESSION_COOKIE_SECURE i CSRF_COOKIE_SECURE).

- **Walidacja nagłówka Host**

  Django waliduje nagłówki Host przeciwko ustawieniu ALLOWED_HOSTS, aby zapobiec różnym atakom, takim jak CSRF i zatruwanie cache. W przypadku użycia serwera proxy odwrotnego należy sprawdzić konfigurację USE_X_FORWARDED_HOST .

- **Bezpieczne zarządzanie sesjami**

  Django oferuje zabezpieczenia, takie jak ścisłe sprawdzanie identyfikatorów sesji oraz możliwość skonfigurowania sesji do wygasania po pewnym czasie nieaktywności. Należy zabezpieczyć sesje, używając ustawień SESSION_COOKIE_SECURE i CSRF_COOKIE_SECURE.

- **Uwierzytelnianie i autoryzacja**

  Django posiada wbudowany system uwierzytelniania i autoryzacji, który można łatwo rozszerzać i dostosowywać.

- **Treści przesyłane przez użytkowników**

  Jeśli strona akceptuje pliki przesyłane przez użytkowników, warto ograniczyć ich rozmiar i typy, aby zapobiec atakom typu DoS oraz wykonaniu złośliwego kodu oraz serwować pliki przesyłane przez użytkowników z odrębnej domeny, aby zapobiec atakom XSS.

Dodatkowe tematy bezpieczeństwa:

Warto także pamiętać o innych kwestiach bezpieczeństwa, takich jak:

- Umieszczenie kodu Pythona poza katalogiem głównym serwera webowego.
- Ograniczenie liczby żądań uwierzytelniających, aby zapobiec atakom brute-force.
- Zachowanie tajności SECRET_KEY.
- Zabezpieczenie dostępu do systemów cache'owania i baz danych.
- Zapoznanie się z listą Top 10 OWASP i zaleceniami Mozilli dotyczącymi bezpieczeństwa aplikacji webowych. [10]

2.2.2. Zalecenia, standardy i najlepsze praktyki bezpieczeństwa w React

React to popularna biblioteka JavaScript do budowy interfejsów użytkownika, opracowana przez Facebook. Chociaż React skupia się głównie na warstwie prezentacji, istnieje wiele praktyk, które można wdrożyć, aby zapewnić bezpieczeństwo aplikacji.

- **Ochrona przed atakami typu XSS (Cross-Site Scripting)**

  React automatycznie eskaluje wszelkie wyjścia HTML, co pomaga zapobiegać atakom XSS. Najlepszą praktyką jest unikanie używania dangerouslySetInnerHTML, chyba że jest to absolutnie konieczne, i upewnienie się, że dane są prawidłowo sanitowane przed wstrzyknięciem.

- **Bezpieczne zarządzanie stanem i danymi**

  Należy upewnić się, że wszelkie dane wrażliwe są odpowiednio zarządzane i przechowywane. Najlepszą praktyką jest korzystanie z bezpiecznych metod przechowywania danych, takich jak localStorage i sessionStorage z odpowiednimi zabezpieczeniami, a także z odpowiednich bibliotek do zarządzania stanem, takich jak Redux, które oferują dodatkowe mechanizmy.

- **Uwierzytelnianie i autoryzacja**

  Zalecane jest, aby implementować mechanizmy uwierzytelniania i autoryzacji w celu kontrolowania dostępu do różnych części aplikacji. Najlepszą praktyką jest korzystanie z tokenów JWT (JSON Web Tokens) do uwierzytelniania użytkowników i zabezpieczania komunikacji między frontendem a backendem.

- **Bezpieczne zarządzanie zależnościami**

  Zalecane jest regularnie aktualizowanie zależności i biblioteki używanej w projekcie React, aby korzystać z najnowszych poprawek bezpieczeństwa. Najlepszą praktyką jest używanie narzędzi takich jak npm audit i yarn audit, aby skanować zależności pod kątem znanych luk bezpieczeństwa i natychmiast aktualizować wszelkie podatne pakiety.

- **Bezpieczna konfiguracja aplikacji**

  Należy upewnić się, że konfiguracja aplikacji jest bezpieczna i nie zawiera poufnych danych w kodzie źródłowym. Najlepszą praktyką jest przechowywanie kluczy API i innych poufnych danych w zmiennych środowiskowych i używanie narzędzi do ich zarządzania, takich jak dotenv.

- **Bezpieczna komunikacja**

  Należy używać protokołu HTTPS do zabezpieczenia komunikacji między frontendem a backendem. Najlepszą praktyką jest skonfigurowanie certyfikatów SSL/TLS i wymuszenie użycia HTTPS, aby chronić dane przesyłane między klientem a serwerem. [5]

2.2.3. Zalecenia, standardy i najlepsze praktyki bezpieczeństwa dla bazy PostgreSQL

- **Zarządzanie uprawnieniami**

  Należy stosować zasadę najmniejszych uprawnień (least privilege), przyznając użytkownikom tylko te uprawnienia, które są niezbędne do wykonania ich zadań oraz unikać używania superużytkowników (np. roli postgres) do codziennych operacji. Ważne jest także, aby używać silnych haseł i regularnie je zmieniać. Można także rozważyć konfigurację mechanizmów autoryzacji na podstawie certyfikatów.

- **Zabezpieczenia sieciowe**

  Należy skonfigurować odpowiednie reguły firewalla, aby ograniczyć dostęp do serwera bazy danych tylko do zaufanych źródeł IP oraz używać szyfrowania SSL/TLS do zabezpieczenia komunikacji między klientami a bazą danych.

- **Aktualizacje i patche**

  Ważne jest, aby monitorować i stosować regularne aktualizacje oraz łatki bezpieczeństwa dostarczane przez PostgreSQL. Aktualizować również system operacyjny i oprogramowanie zależne.

- **Audytowanie i monitorowanie**

  Należy włączyć audytowanie zdarzeń w PostgreSQL, aby rejestrować próby dostępu, zmiany w danych i inne ważne operacje. Warto także monitorować wydajność bazy danych oraz analizować logi w celu wczesnego wykrywania podejrzanych aktywności.

- **Zarządzanie danymi i backupami**

  Należy regularnie tworzyć i przechowywać kopie zapasowe danych, zgodnie z zasadą 3-2-1 (trzy kopie danych, na dwóch nośnikach, jedna poza lokalizacją). Dodatkowo należy stosować odpowiednie mechanizmy szyfrowania danych przechowywanych w bazie, szczególnie danych wrażliwych. [11]

3. Faza projektowania rozwiązań

3.1. Przegląd wymagań bezpieczeństwa

W ramach naszego projektu, istotnym elementem zapewnienia bezpieczeństwa aplikacji jest skuteczne zabezpieczenie komunikacji między backendem a frontendem, a także zapewnienie bezpiecznego mechanizmu logowania użytkowników.

- **Walidacja danych**

  Walidacja danych jest kluczowym elementem zapewnienia bezpieczeństwa i integralności aplikacji webowej. W naszym projekcie walidacja jest realizowana na trzech poziomach: frontend (React), backend (Django) i baza danych (PostgreSQL).

  1) **Walidacja na poziomie frontendu (React)**

     Walidacja na poziomie frontendu ma na celu zapewnienie, że dane wprowadzane przez użytkowników są poprawne zanim zostaną przesłane do serwera. Dzięki temu użytkownik może natychmiast zobaczyć błędy w formularzach i je poprawić. Jeśli dane nie spełniają wymagań (np. słowo zawiera polskie znaki), formularz wyświetla komunikat błędu i blokuje przesłanie danych. [12]

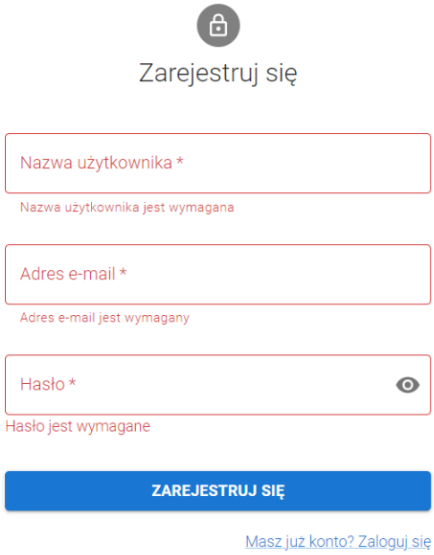
     Do zarządzania formularzami w React można wykorzystać bibliotekę "Formik". Do walidacji schematów w JavaScript można wykorzystać bibliotekę "Yup".

     Nasz projekt obejmuje jedynie podstawową walidacje, czyli sprawdzanie formatu nazwy użytkownika oraz adresu email i sprawdzanie czy pola nie są puste. Skrypt tej walidacji pokazano na rysunku 3.1, a przykład działania tejże walidacji na rysunkach 3.2 oraz 3.3.

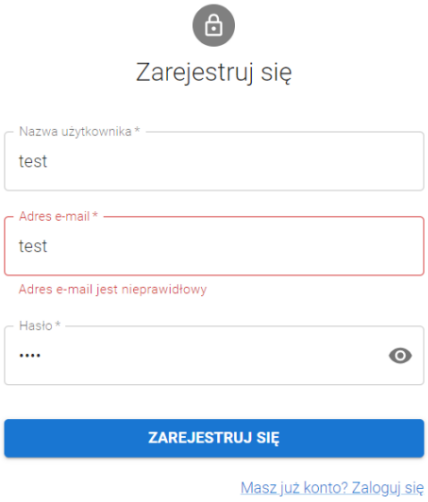
     Korzystne byłoby rozszerzenie walidacji poprzez wymaganie dłuższego i bardziej skomplikowanego hasła, informowanie przy rejestracji, że użytkownik o danej nazwie jest już zajęty, albo że użytkownik z danym adresem email już jest w bazie.

```
const validate = () => {
  const newErrors = {};
  if (!username) newErrors.username = "Nazwa użytkownika jest wymagana";
  if (!email) {
    newErrors.email = "Adres e-mail jest wymagany";
  } else if (!/\S+@\S+\.\S+/.test(email)) {
    newErrors.email = "Adres e-mail jest nieprawidłowy";
  }
  if (!password) newErrors.password = "Hasło jest wymagane";
  return newErrors;
};
```

Rysunek 3.1: Przykład walidacji z poziomu frontendu.



Rysunek 3.2: Przykład 1 działania walidacji w naszej aplikacji.



Rysunek 3.3: Przykład 2 działania walidacji w naszej aplikacji.

2) **Walidacja na poziomie backendu (Django)**

Walidacja na backendzie jest niezbędna, aby zapewnić, że dane przetwarzane przez serwer są poprawne i bezpieczne, nawet jeśli walidacja na froncie została pominięta lub obejście przez złośliwego użytkownika. Jeśli dane nie spełniają wymagań backendu (np. API otrzymuje niepoprawne dane), Django zwraca komunikat błędu. [12]

Do tworzenia i walidacji formularzy w Django warto skorzystać z narzędzia "Django Forms". Do zarządzania walidacją danych aplikacji na poziomie modelu warto wykorzystać "Django Models" [13]. Przykład walidacji w formularzach ukazano na rysunku 3.4.

```
from django import forms
from .models import Word

class WordForm(forms.ModelForm):
    class Meta:
        model = Word
        fields = ['word', 'definition']

    def clean_word(self):
        word = self.cleaned_data.get('word')
        if not word.isalpha():
            raise forms.ValidationError('Word must contain only alphabetic characters.')
        return word
```

Rysunek 3.4: Przykład walidacji w formularzach [13].

W przypadku naszego projektu została zaimplementowana jedynie walidacja hasła wykorzystując wbudowane funkcje biblioteki Django, co ukazano na rys. 3.5

```
# Validate password
try:
    validate_password(password)
except ValidationError as e:
    return Response({'detail': e.messages}, status=status.HTTP_400_BAD_REQUEST)
```

Rysunek 3.5: Walidacja hasła.

Ten fragment kodu używa wbudowanej metody validate_password dostępnej w Django do sprawdzenia poprawności hasła. Jeśli hasło nie

spełnia wymagań określonych przez walidatory, zostanie przechwycony wyjątek ValidationError. W takim przypadku zwracana jest odpowiedź HTTP 400 (Bad Request) zawierająca szczegóły błędów walidacji.

Kod biblioteki używanej w Django, definiujący metodę ”validate_password”, która została przez nas wykorzystana przedstawiono na rys. 3.6

```
def validate_password(password, user=None, password_validators=None):
    """
    Validate that the password meets all validator requirements.

    If the password is valid, return ``None``.
    If the password is invalid, raise ValidationError with all error messages.
    """
    errors = []
    if password_validators is None:
        password_validators = get_default_password_validators()
    for validator in password_validators:
        try:
            validator.validate(password, user)
        except ValidationError as error:
            errors.append(error)
    if errors:
        raise ValidationError(errors)
```

Rysunek 3.6: Definicja metody użytej do walidacji haseł.

Zastosowane walidatory hasła wbudowane w Django przedstawiono na rys. 3.7

```
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
```

Rysunek 3.7: Zastosowane walidatory hasła.

UserAttributeSimilarityValidator: walidator sprawdza, czy hasło jest zbyt podobne do atrybutów użytkownika: nazwa użytkownika, adres e-mail. Jest to przydatne, aby zapobiec sytuacjom, w których użytkownicy ustawiają hasło zbyt łatwe do odgadnięcia na podstawie ich publicznie dostępnych informacji.

MinimumLengthValidator: walidator sprawdza, czy hasło ma minimalną długość określoną w konfiguracji. Jest to podstawowy krok w zapewnieniu siły hasła.

CommonPasswordValidator: walidator sprawdza, czy hasło nie jest jednym z powszechnie używanych haseł. Lista tych haseł jest zawarta w Django i można ją dostosować. Zapobiega to używaniu haseł, które są zbyt proste i popularne.

NumericPasswordValidator: walidator sprawdza, czy hasło nie składa się wyłącznie z cyfr. Hasła składające się tylko z cyfr są łatwiejsze do odgadnięcia i mniej bezpieczne, więc ten walidator pomaga zapobiec używaniu takich haseł.

3) **Walidacja na poziomie bazy danych (PostgreSQL)**

Walidacja na poziomie bazy danych zapewnia integralność danych przechowywanych w bazie. W przypadku aplikacji korzystającej z relacyjnej bazy danych, można użyć klauzul CHECK do dodatkowego zabezpieczenia i zapewnienia integralności danych. Jeśli dane nie spełniają wymagań bazy danych, operacja zapisu jest odrzucana. Przykładową klauzulę CHECK pokazano na rysunku 3.8.

```
CREATE TABLE Words (
    id SERIAL PRIMARY KEY,
    word VARCHAR(255) NOT NULL,
    definition TEXT NOT NULL,
    CHECK (word ~ '^[A-Za-z]+$')
);
```

Rysunek 3.8: Przykład klauzuli CHECK [14].

W przypadku naszej aplikacji zastosowano statement **"Generated always as identity"** (rys. 3.9) oraz klauzulę SQL **"Unique Constraint"** (rys. 3.10)

```
ALTER TABLE public."user" ALTER COLUMN id ADD GENERATED ALWAYS AS IDENTITY (
    SEQUENCE NAME public.user_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);
```

Rysunek 3.9: "Generated always as identity".

Polecenie definiuje kolumnę id jako typ typ integer, który jest generowany automatycznie (GENERATED ALWAYS AS IDENTITY). Oznacza to, że system bazodanowy automatycznie, sekwencyjnie przypisuje unikalne wartości do kolumny id podczas dodawania nowych rekordów. Uniemożliwia to ręczne wpisanie takich rekordów, co oznacza również, że nie trzeba się martwić o konflikty lub duplikaty identyfikatorów przy wstawianiu nowych rekordów do tabeli.

```
ALTER TABLE ONLY public."user"
    ADD CONSTRAINT user_username_key UNIQUE (username);
ALTER TABLE ONLY public."user"
    ADD CONSTRAINT user_email_key UNIQUE (email);
```

Rysunek 3.10: "Unique Constraint".

Ta operacja dodaje warunek UNIQUE dla kolumny email oraz username, co oznacza, że każdy adres email i nazwa użytkownika w tej tabeli muszą być unikalne. Zapobiega to powtarzaniu się tych samych nazw i adresów email dla różnych użytkowników.

Podczas tworzenia naszej aplikacji, integracja walidacji na wszystkich trzech poziomach jest kluczowa.Użytkownik najpierw wprowadza dane na stronie internetowej, gdzie walidacja frontendowa sprawdza ich poprawność. Następnie dane są przesyłane do backendu, gdzie przechodzi przez kolejną warstwę walidacji w

Django. Ostatecznie, przed zapisaniem danych w bazie danych, walidacja na poziomie bazy danych zapewnia ich integralność.

- **Zabezpieczenie komunikacji**

  W celu zapewnienia poufności i integralności danych przesyłanych między serwerem a przeglądarką istotne jest wykorzystanie protokołu HTTPS. Protokół ten zapewnia szyfrowanie danych w transmisji, uniemożliwiając ich przechwycenie i odczytanie przez osoby trzecie, co zapewnia poufność i integralność danych przesyłanych między klientem a serwerem. Należy skonfigurować serwer i frontend tak, aby korzystały z certyfikatów SSL/TLS, gwarantujących bezpieczną komunikację. [15]

  W naszym projekcie niestety nie zostanie ten element wdrożony ze względu na ograniczone zasoby czasowe.

- **Zabezpieczenie funkcji Logowania**

  Aby zapewnić bezpieczne logowanie użytkowników, planujemy zaimplementować mechanizm uwierzytelniania oparty na tokenach JWT (JSON Web Token). JWT to standardowy format otwartego, samoopisującego się i kompaktowego tokena używanego do bezpiecznej transmisji informacji między stronami aplikacji w formie JSON. Charakteryzuje się on kompaktowym formatem, który może być łatwo przesyłany przez sieć między klientem a serwerem. Zawiera wszystkie niezbędne informacje o użytkowniku lub sesji, co eliminuje potrzebę przechowywania stanu sesji na serwerze. Może być podpisany cyfrowo podpisany w celu weryfikacji integralności danych. Może także być szyfrowany, jeśli wymaga tego bezpieczeństwo.

  Po poprawnym uwierzytelnieniu, serwer będzie generował token JWT, który będzie zawierał informacje o zalogowanym użytkowniku oraz okresie ważności. Ten token będzie przesyłany przez klienta w nagłówku żądania przy każdej kolejnej interakcji z serwerem. Dzięki temu, każde żądanie będzie mogło być jednoznacznie zidentyfikowane i uwierzytelnione przez serwer, zapewniając dostęp tylko do odpowiednich zasobów dla zalogowanego użytkownika. [16]

  Implementacja uwierzytelniania użytkowników zostanie wykonana przy użyciu wbudowanych mechanizmów Django django.contrib.auth. Warto zaznaczyć,

że zamiast przechowywać tokeny JWT w localStorage, co jest podatne na ataki XSS, lepszym rozwiązaniem jest przechowywanie ich w ciasteczkach (cookies) ustawionych z flagą HttpOnly oraz Secure. Zabezpieczając ciasteczka w ten sposób, zapobiegamy ich dostępności z poziomu JavaScript oraz wymuszamy ich przesyłanie tylko przez bezpieczne połączenia HTTPS.

Tokeny te są wygodne, jednak istnieją bezpieczniejsze metody, takie jak oAuth2, które oferują bardziej zaawansowane mechanizmy uwierzytelniania i autoryzacji.

- **Blokada czasowa i dodatkowe zabezpieczenia**

  Mimo iż obecnie nie implementujemy blokady czasowej po kilku nieudanych próbach logowania, warto rozważyć wprowadzenie tego mechanizmu, aby zwiększyć bezpieczeństwo aplikacji. Dodatkowe praktyki, które warto byłoby rozważyć to wymuszanie zmiany hasła co pewien czas; implementacja uwierzytelniania dwuskładnikowego (2FA); czas odpowiedzi nie zdradzająca, czy konto nie istnieje; działania, jeśli użytkownik loguje się z nieznanej lokalizacji; powiadomienie użytkownika, jeśli jego hasło istnieje w bazach skradzionych haseł, czy też przechowywanie danych osobowych (e-mail, telefon) w postaci zaszyfrowanej. W naszej aplikacji walidacja hasła na backendzie jest wbudowana a dodatkowo zaimplementowane zostało jedynie hashowanie haseł w bazie danych.

- **Bezpieczne zasady przechowywania danych**

  Zasada najmniejszego uprzywilejowania (Least Privilege)

  Każdy użytkownik powinien mieć przyznane tylko te uprawnienia, które są niezbędne do wykonania swoich obowiązków. Nie należy nadawać zbyt szerokich uprawnień, aby zminimalizować ryzyko nieautoryzowanego dostępu lub uszkodzenia danych. [17]

  Użycie oddzielnych połączeń (Separate Connections/Users)

  Każdy użytkownik systemu powinien mieć swoje unikalne połączenie do bazy danych z indywidualnymi danymi uwierzytelniającymi. Dzięki temu można śledzić i zarządzać dostępem do danych oraz zapobiegać konfliktom zasobów.

Utworzenie różnych schematów lub oddzielnych baz

W zależności od potrzeb i charakteru danych, warto rozważyć umieszczanie różnych typów danych w oddzielnych schematach lub bazach danych. To pozwala na łatwiejsze zarządzanie uprawnieniami dostępu oraz izolację danych o różnym stopniu poufności lub zastosowań.

Ograniczenia integralnościowe (Integrity Constraints)

W bazach danych należy stosować ograniczenia integralnościowe, takie jak klucze główne, unikalne, oraz klucze obce, aby zapewnić poprawność danych i uniknąć niezgodności. Ograniczenia te pomagają również w utrzymaniu spójności danych w całym systemie.

Używanie GUID zamiast liczbowych identyfikatorów

Aktualnie w naszej aplikacji stosujemy identyfikatory typu int dla użytkowników i innych zasobów. Jednak bardziej bezpieczne i unikalne są identyfikatory GUID, które minimalizują ryzyko przewidywalności i kolizji.

- **Zabezpieczenie Publicznego API**

Ze względów bezpieczeństwa przeglądarki ograniczają żądania HTTP cross-origin inicjowane przez skrypty. Na przykład, fetch() i XMLHttpRequest stosują politykę tego samego źródła (same-origin policy). Oznacza to, że aplikacja internetowa używająca tych API może żądać zasobów tylko z tego samego źródła, z którego została załadowana.

Celem ograniczenia dostępu do publicznego interfejsu API tylko dla autoryzowanych użytkowników, planujemy wprowadzić mechanizm uwierzytelniania oparty na kluczu API. Każdy użytkownik, który chce korzystać z publicznego API, będzie musiał uzyskać klucz API poprzez proces autoryzacji. Ten klucz będzie następnie wykorzystywany do uwierzytelniania zapytań do API, zapewniając, że tylko uprawnione aplikacje lub użytkownicy mają dostęp do jego funkcji. Do kontrolowania dostępu do API z zewnętrznych domen planujemy zastosować konfigurację CORS (Cross-Origin Resource Sharing) w Django za pomocą django-cors-headers.

Cross-Origin Resource Sharing (CORS) jest mechanizmem opartym na nagłówkach HTTP, który pozwala serwerowi wskazać dowolne źródła (domenę,

schemat lub port) inne niż jego własne, z których przeglądarka powinna zezwolić na ładowanie zasobów. CORS opiera się również na mechanizmie, w którym przeglądarki wykonują zapytanie wstępne do serwera hostującego zasób pochodzący z innego źródła, aby sprawdzić, czy serwer zezwoli na faktyczne żądanie. W tym zapytaniu wstępnym przeglądarka wysyła nagłówki wskazujące metodę HTTP i nagłówki, które będą używane w faktycznym żądaniu. [18]

W naszej aplikacji webowej Django i React funkcjonują jako oddzielne aplikacje, które są ze sobą integrowane poprzez API. Dzięki odpowiednio skonfigurowanym nagłówkom CORS, serwer Django naszej aplikacji zezwala na realizację żądania z serwera Reacta, mimo że pochodzi ono z innego źródła. Mechanizm ten jest istotny z punktu bezpieczeństwa, ponieważ zapobiega to możliwości ataków typu cross-site scripting (XSS) oraz cross-site request forgery (CSRF), które mogą wystąpić, gdy nieautoryzowane strony próbują wykorzystać skrypty lub manipulować żądaniami HTTP między różnymi domenami.

- **Ochrona przed CSRF**

  Ochrona przed CSRF w Django zostanie zrealizowana za pomocą wbudowanych mechanizmów ochrony przed CSRF w Django, które są domyślnie włączone. W praktyce oznacza to, że Django automatycznie dodaje tokeny CSRF do formularzy HTML, co chroni aplikację przed tego typu atakami.

3.2. Weryfikacja architektury projektu

Planowane funkcjonalności:

- **Rejestracja i logowanie użytkowników**

  Możliwość tworzenia konta użytkownika. Logowanie za pomocą e-maila i hasła.

- **Dodawanie i zarządzanie słówkami**

  Opcja dodawania nowych słówek wraz z ich tłumaczeniem, a także ich edycja oraz możliwość usuwania. Kategorie umożliwiające organizowanie słówek tematycznie.

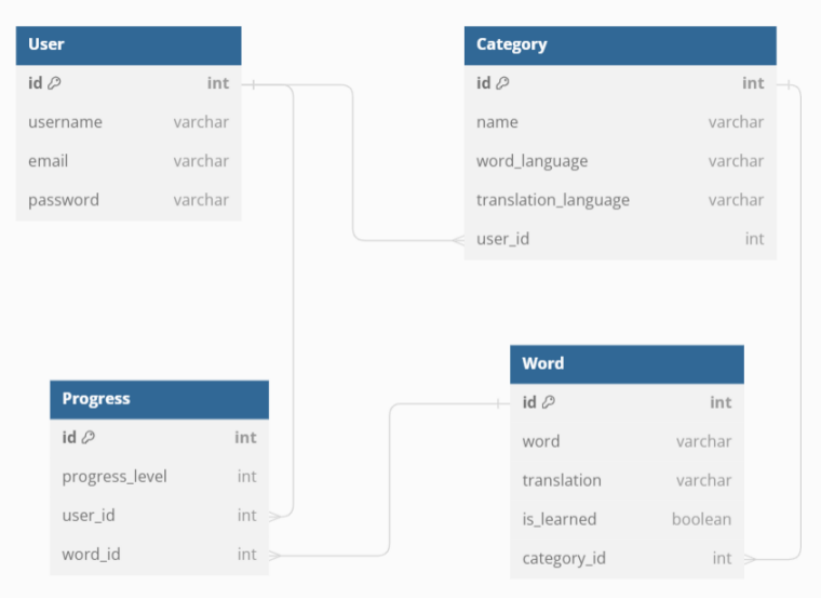
- **Bezpieczne przechowywanie danych**

  Wbudowane oraz zaimplementowane mechanizmy szyfrowania danych użytkownika.

- **Zarządzanie dostępem do zasobów**

  W ramach architektury projektu przewidziano mechanizmy zarządzania dostępem do zasobów. Przykładowo, katalog słówek jest dostępny tylko dla zalogowanych użytkowników. W przypadku, gdy użytkownik nie jest zalogowany i próbuje uzyskać dostęp do chronionego zasobu, zostanie przekierowany na stronę logowania. Jest to kluczowy element zapewnienia bezpieczeństwa i integralności aplikacji.

Na rysunku 3.11 przedstawiono diagram bazy danych, który planowano zastosować, ale ze względu na ograniczone zasoby czasowe konieczna była rezygnacja z encji Progress.



Rysunek 3.11: Diagram bazy danych, który planowaliśmy zastosować.

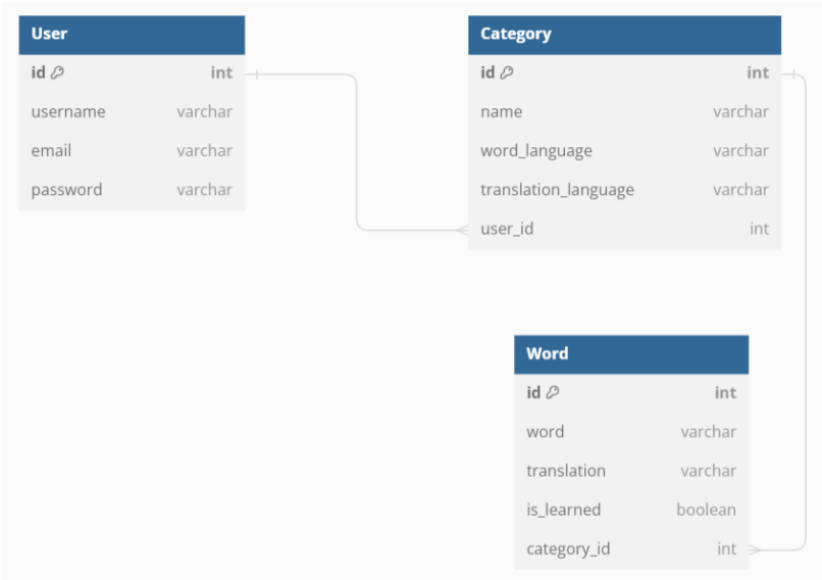
Encja Progress czyli postęp w nauce danego słówka miała umożliwiać użytkownikowi, który uczy się słówek z danej kategorii na zaznaczenie czy zna słówko czy jeszcze nie. Dodatkowo po 5 zaznaczeniach, że dane słówko już zna i zostało przez niego utwalone, to słówko znikało z zestawu i nie pokazywało się więcej w fiszkach.

Planowano również dodać opcję wyświetlania poziomu nauczenia obok danego słówka w poniższy sposób:

```
ProgressLevel INT DEFAULT 0 CHECK (ProgressLevel IN (0, 20, 40, 60, 80, 100))
```

gdzie: default 0 - to wartość początkowa pola, która później byłaby zwiększana co 20, aż do 100.

Na rysunku 3.12 przedstawiono diagram bazy danych, który zastosowano w naszej aplikacji.



Rysunek 3.12: Diagram bazy danych zastosowany w naszej aplikacji.

4. Faza tworzenia aplikacji

4.1. Implementacja backendu i frontendu

- **Obsługa autoryzacji i autentykacji**

  Implementacja mechanizmów, które przekierowują użytkowników na stronę logowania, jeśli próbują uzyskać dostęp do chronionych zasobów (np. katalogu słówek). W Django za obsługę autoryzacji i autentykacji odpowiadają wbudowane funkcje, które umożliwiają łatwe wdrożenie tego rodzaju zabezpieczeń. W React za zarządzanie stanem autoryzacji użytkownika można wykorzystać Context API lub Redux. Przykładowo, katalog słówek jest dostępny tylko dla zalogowanych użytkowników. Używając komponentu ProtectedRoute, sprawdzamy, czy użytkownik jest zalogowany. Jeśli nie, użytkownik zostaje przekierowany na stronę logowania. Dzięki temu mechanizmowi tylko autoryzowani użytkownicy mają dostęp do chronionych zasobów.

- **Implementacja JWT**

  JWT składa się z trzech części:

  - **Nagłówek (Header)** który zawiera typ tokena i algorytm użyty do podpisania tokena.
  - **Ładunek (Payload)** który zawiera zestaw danych, które reprezentują deklaratywne stwierdzenia o jednostce (np. użytkowniku) oraz dodatkowe metadane.
  - **Podpis (Signature)** który jest podpisem cyfrowym obliczonym na podstawie kodowania base64 zakodowanego nagłówka, zakodowanego ładunku oraz sekretnego klucza lub certyfikatu.Służy do weryfikacji, że token nie został zmieniony w trakcie transmisji. [16]

  Konfiguracja krok po kroku:

  1) Instalacja bibliotek Django Rest Framework i Simple-JWT (rys. 4.13)
  2) Dodanie modułów w ustawieniach projektu (settings.py) (rys. 4.14)
  3) Konfiguracja JWT (settings.py) (rys. 4.15)

4) Dodanie punktów końcowych dla tokenu JWT (urls.py) (rys. 4.16)

5) Przypisanie uwierzytelnienia dla widoków API (views.py) (rys. 4.17)

```
pip install django djangorestframework djangorestframework_simplejwt
```

Rysunek 4.13: Instalacja bibliotek.

```
INSTALLED_APPS = [
    ...,
    'rest_framework',
    'rest_framework_simplejwt',
    ...,
]
```

Rysunek 4.14: Dodanie modułów.

```
from datetime import timedelta

REST_FRAMEWORK = {
    'DEFAULT_PERMISSION_CLASSES': (
        'rest_framework.permissions.IsAuthenticated',
    ),
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
}

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),
    'SLIDING_TOKEN_REFRESH_LIFETIME': timedelta(days=1),
    'SLIDING_TOKEN_LIFETIME': timedelta(days=30),
    'SLIDING_TOKEN_REFRESH_LIFETIME_LATE_USER': timedelta(days=1),
    'SLIDING_TOKEN_LIFETIME_LATE_USER': timedelta(days=30),
}
```

Rysunek 4.15: Konfiguracja JWT.

```
from django.urls import path, include
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView

urlpatterns = [
    ...,
    path('api/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
    path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
    ...,
]
```

Rysunek 4.16: Dodanie punktów końcowych.

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from rest_framework_simplejwt.authentication import JWTAuthentication

class HomeView(APIView):
    authentication_classes = [JWTAuthentication]
    permission_classes = [IsAuthenticated]

    new *
    def get(self, request):
        content = {'message': "Successfully connected to the API"}
        return Response(content)
```

Rysunek 4.17: Przypisanie uwierzytelnienia.

- **Hashowanie haseł przechowywanych w bazie danych**

Do tworzenia i sprawdzania haseł podczas logowania i rejestracji użytkownika użyto metod set_password i check_password, które są integralną częścią Django i służą do hashowania i porównywania haseł, co przedstawiono na rys. 4.18

```
from django.contrib.auth.hashers import make_password, check_password

  def set_password(self, raw_password):
      self.password = make_password(raw_password)


  def check_password(self, raw_password):
      return check_password(raw_password, self.password)
```

Rysunek 4.18: Metody hashowania i porównywania haseł.

Metoda make_password(raw_password) jest używana do wygenerowania zabezpieczonego hasła na podstawie podanego raw_password. Django domyśl-

nie używa algorytmu PBKDF2 (Password-Based Key Derivation Function 2) z HMAC (Hash-based Message Authentication Code) i zastosowaniem SHA-256 jako funkcji haszującej. Django generuje unikalną solę dla każdego hasła, które jest haszowane. Sól jest dodawana do hasła przed zastosowaniem algorytmu haszowania. Algorytm PBKDF2 używany przez Django wykonuje wielokrotne iteracje, aby zwiększyć czas potrzebny do hashowania hasła, co zapewnia większe bezpieczeństwo. Metoda check_password(raw_password, encoded_password) porównuje podane hasło raw_password z zakodowanym hasłem encoded_password. W ten sposób można bezpośrednio zwalidować hasło bez przechowywania go w postaci zwykłego tekstu.

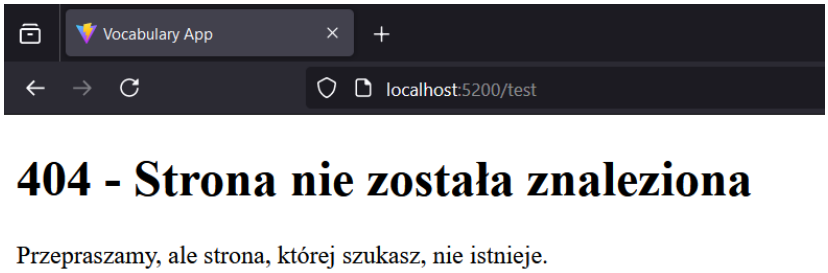
Zahashowane hasła w bazie danych ukazano na rys. 4.19

id [PK] integer	username character varying (50)	email character varying (80)	password character varying (255)
1	User1234	email@onet.com.pl	pbkdf2_sha256$720000$pTC39pfGb6IPqmlcRJWkax$ZCjC29ca3pP9VtaT4v/Qyd7Yxfqz3O6OG1GGWc7J4u...
2	Marcin	mail@mail.com	pbkdf2_sha256$720000$hZVaUMFA60PhIXfHnyXmfc$nnfnFcijitqyI2tCvW78+sFFFSecuTRTRSXYuW/Maf0=
3	Marian	mail4@mail.com	pbkdf2_sha256$720000$nj1XWJEtv2Lt6SQiOh1qsM$dLcEc/shkF9HQKiLOmgoYmiHvvzSCTGbSRKQXAiTfL...

Rysunek 4.19: Zahashowane hasła.

- **Obsługa błędnych adresów URL**

Zaimplementowano mechanizmy obsługi błędnych adresów URL. Jeśli użytkownik poda adres, który nie istnieje (np. /test), aplikacja wyświetli odpowiedni komunikat informujący, że taka strona nie istnieje. Jest to realizowane za pomocą systemu routingu w Django, który zwraca odpowiedni status HTTP (np. 404 Not Found) i wyświetla stronę błędu. Przykład tego mechanizmu zastosowanego w naszej aplikacji ukazano na rysunku 4.20



Rysunek 4.20: Komunikat o nie znalezieniu strony.

- **Przechowywanie danych - plik .env:**

  Biorąc pod uwagę kwestię umieszczenia bazy danych w chmurze publicznej, istotne jest zabezpieczenie zawartych w niej danych, aby nikt z zewnątrz nie mógł połączyć się z naszą bazą ani aplikacją. W tym celu stworzono plik .env, który zawiera wszystkie dane konfiguracyjne, w tym dane dostępu do bazy danych. Plik ten nie zostanie umieszczony w repozytorium na GitHubie, aby nie był publicznie widoczny. Do połączenia się z bazą danych konieczne będzie posiadanie odpowiedniego ”secret key”, który odpowiada za uwierzytelnianianie i szyfrowanie danych. Jest to kluczowa zmienna środowiskowa, którą również należy przechowywać w pliku ”.env”, co pokazano na rys. 4.21

```
27  # SECURITY WARNING: keep the secret key used in production secret!
28  SECRET_KEY = env('SECRET_KEY')
```

Rysunek 4.21: "Secret key"przechowywany w pliku .env.

Plik „.env” (dot-env) jest powszechnie stosowanym rozwiązaniem do przechowywania plików lub zmiennych środowiskowych używanych do przechowywania konfiguracji i ustawień aplikacji, które mogą zawierać dane takie jak konfiguracja bazy danych, ustawienia serwera, zabezpieczenia i klucze API.

Przechowywanie wrażliwych informacji, takich jak hasła do baz danych, w pliku .env pozwala na ich bezpieczne zarządzanie. Plik ten jest zazwyczaj dodawany do .gitignore, co uniemożliwia przypadkowe umieszczenie tych danych w repozytorium kodu.

Zmienne środowiskowe umożliwiają łatwe zarządzanie konfiguracją aplikacji, bez konieczności modyfikowania kodu. Można mieć różne pliki .env dla różnych środowisk (np. .env.development, .env.production), co ułatwia zarządzanie ustawieniami w różnych fazach cyklu życia aplikacji.

Użycie zmiennych środowiskowych sprawia, że aplikacja jest bardziej przenośna i łatwiejsza do skalowania. Konfiguracje są oddzielone od kodu, co pozwala na łatwe wdrażanie aplikacji w różnych środowiskach.

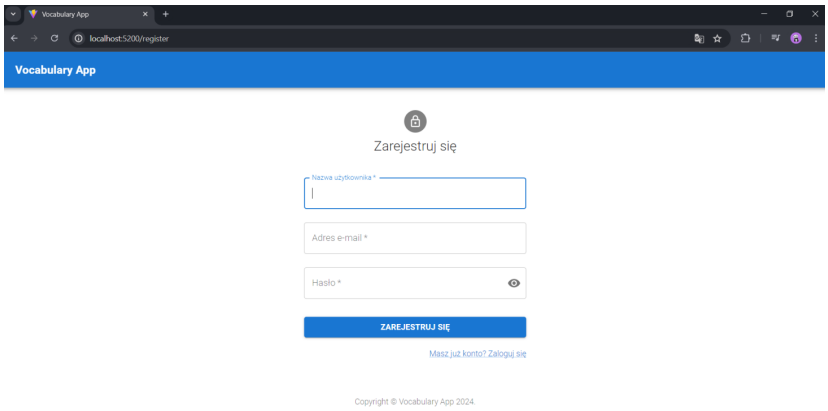
Na rysunku 4.22 przedstawiono fragment kodu z settings.py, który zawiera dane do połączenia z bazą danych PostgreSQL w naszej aplikacji.

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': env('DATABASE_NAME'),
        'USER': env('DATABASE_USER'),
        'PASSWORD': env('DATABASE_PASS'),
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

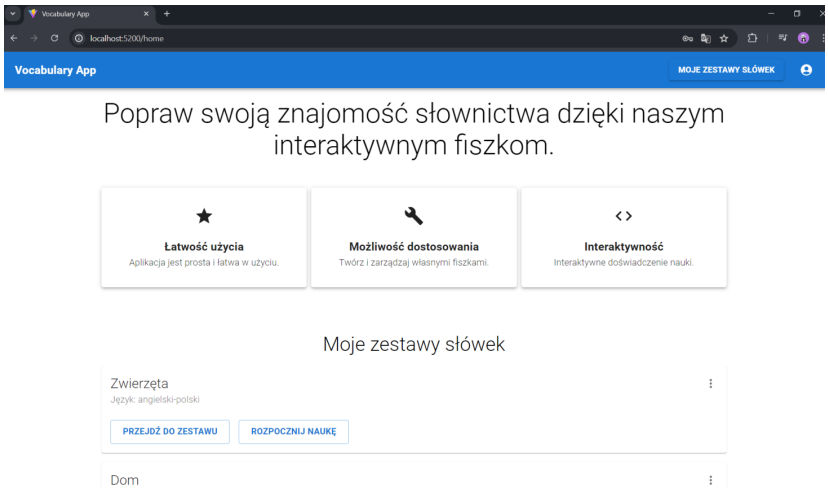
Rysunek 4.22: Fragment kodu z settings.py.

4.2. Projektowanie interfejsu użytkownika

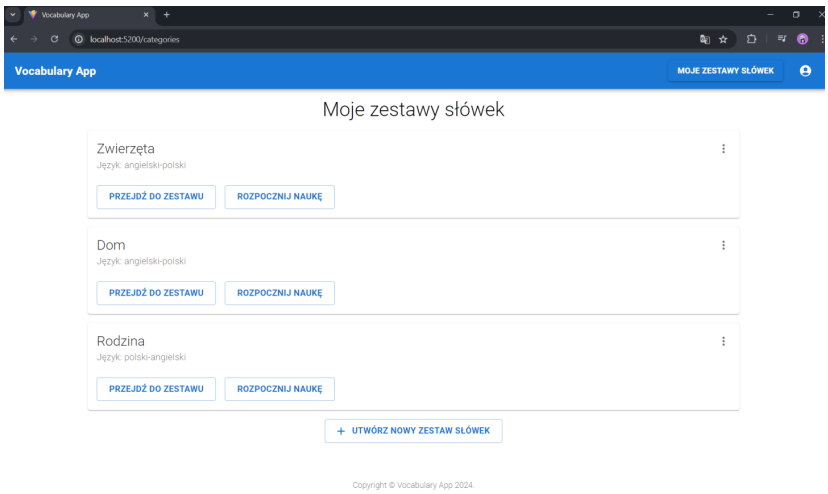
Aplikacja została zaprojektowana z myślą o dostarczeniu użytkownikom narzędzia do efektywnej nauki słówek w języku angielskim. Obejmuje ona interaktywne funkcje, takie jak rejestracja i logowanie (rys. 4.23), dodawanie nowych słów (rys. 4.27), definiowanie ich oraz przeprowadzanie testów wiedzy (rys. 4.29, 4.30). Interfejs użytkownika odgrywa kluczową rolę w ułatwianiu tych procesów oraz zapewnieniu przyjemności z korzystania z aplikacji.



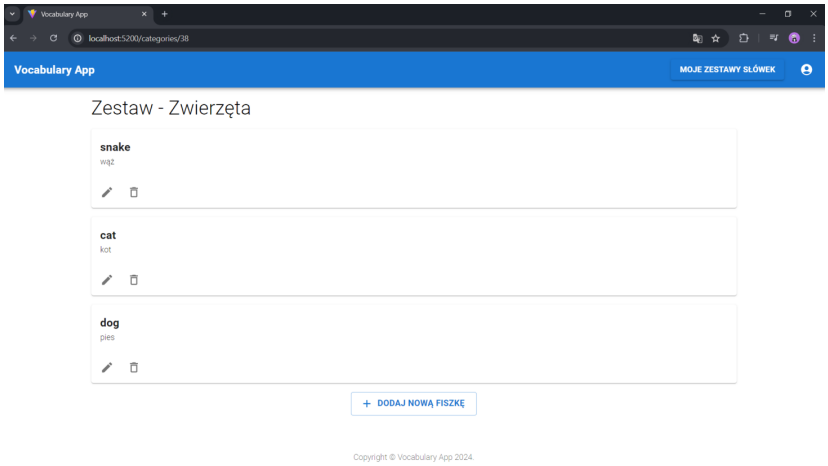
Rysunek 4.23: Okno rejestracji.



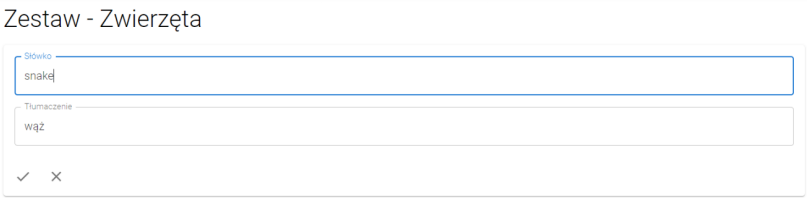
Rysunek 4.24: Strona główna.



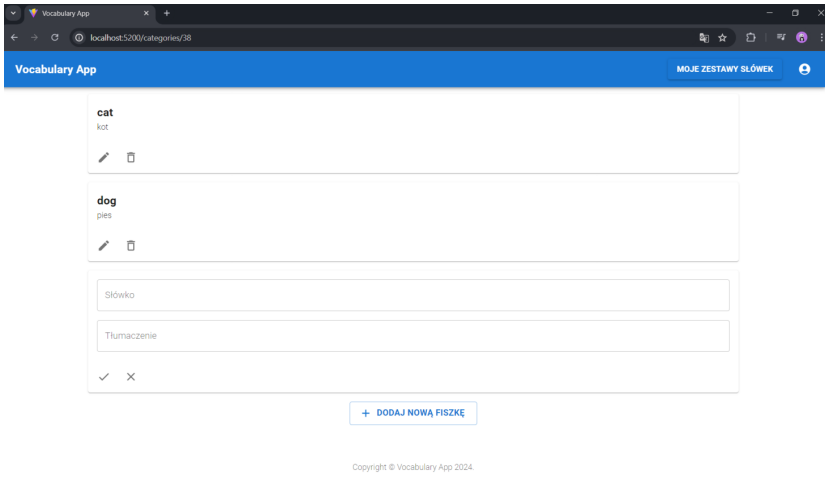
Rysunek 4.25: Kategorie.



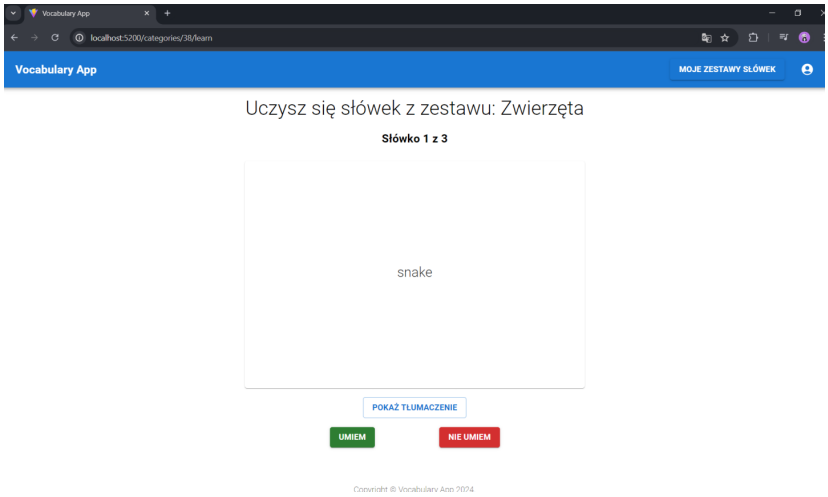
Rysunek 4.26: Przykładowy zestaw.



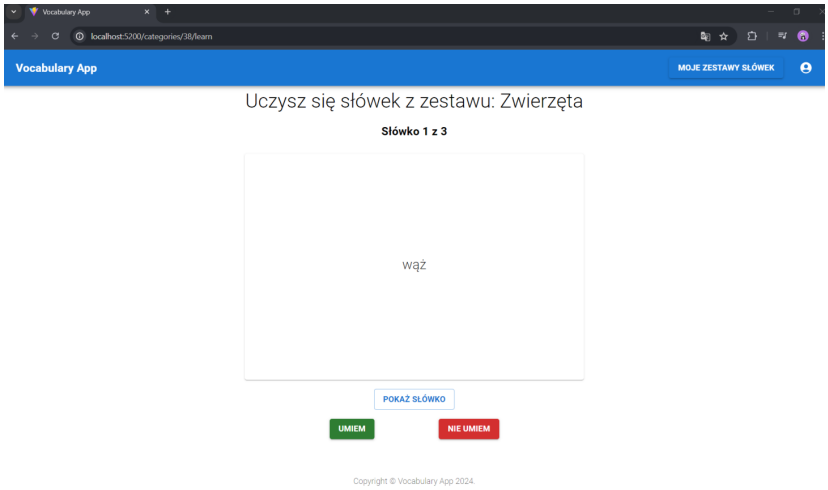
Rysunek 4.27: Tworzenie słówka.



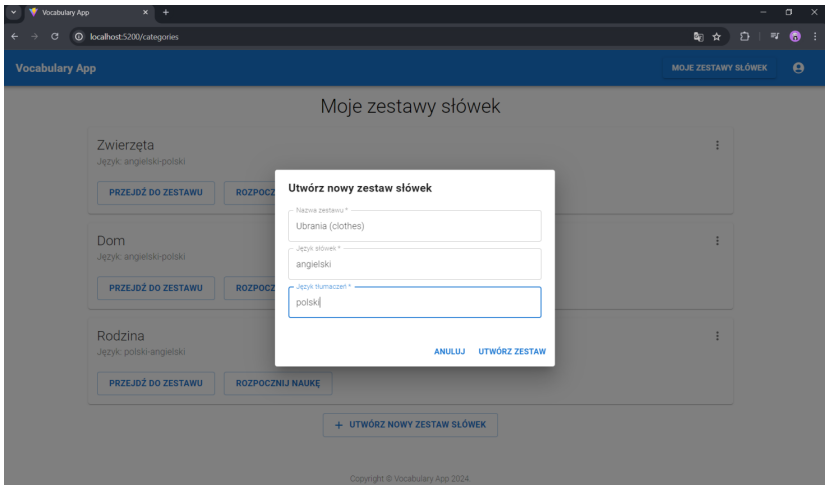
Rysunek 4.28: Możliwość dodania nowej fiszki na dole strony.



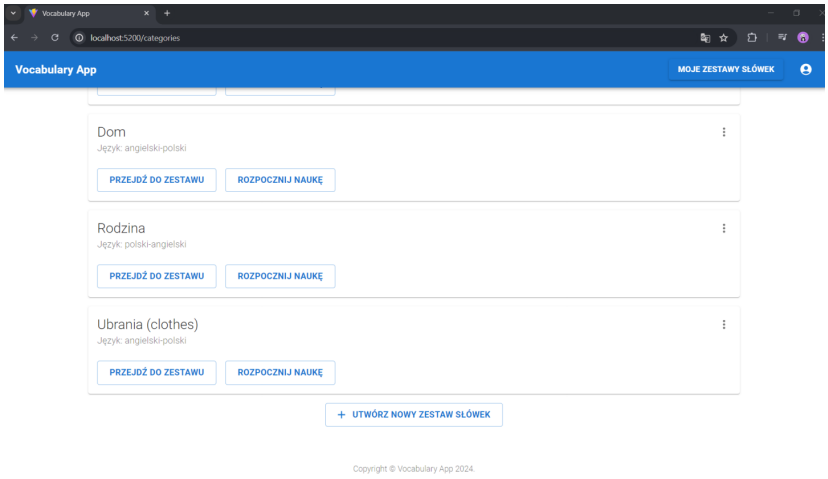
Rysunek 4.29: Okno nauki słówek.



Rysunek 4.30: Okno nauki słówek - tłumaczenie.



Rysunek 4.31: Okno tworzenia nowego zestawu.



Rysunek 4.32: Przegląd utworzonych zestawów słówek.

4.3. Przegląd przepływu danych w aplikacji

Interakcje między użytkownikiem, frontendem stworzonym przy użyciu Reacta, backendem opartym na Django i bazą danych PostgreSQL pokazano na przykładzie diagramu widocznego na rysunku 4.33

Rysunek 4.33: Diagram przepływu danych między frontem, backendem i bazą danych.

Przepływ danych:

1) User -> React Frontend - Użytkownik wprowadza dane lub przegląda istniejące.

2) React Frontend -> Django Backend - Frontend przejmuje dane od użytkownika i wysyła zapytania HTTP do backendu.

3) Django Backend -> PostgreSQL Database - Backend odbiera zapytania HTTP od frontendu, przetwarza je i wysyła zapytania SQL do bazy danych.

4) PostgreSQL Database -> Django Backend - Baza danych odbiera zapytania SQL od backendu i zwraca odpowiedzi.

5) Django Backend -> React Frontend - Backend wysyła odpowiedzi HTTP do frontendu.

6) React Frontend -> User - Frontend renderuje strony i prezentuje je użytkownikowi.

4.4. Kontrola kodu i zapewnienie bezpieczeństwa

- **Zarządzanie dostępem do zasobów**

  Regularna weryfikacja kodu pod kątem prawidłowego zarządzania dostępem do zasobów. Testowanie, czy mechanizmy przekierowywania na stronę logowania działają poprawnie i czy dostęp do chronionych zasobów jest odpowiednio kontrolowany.

- **Obsługa błędnych adresów URL**

  Sprawdzenie, czy obsługa błędnych adresów URL działa prawidłowo i czy aplikacja prawidłowo informuje użytkownika o błędach nawigacyjnych.

- **Konfiguracja CORS dla Django:**

  1) Instalacja biblioteki django-cors-headers (rys. 4.34 lub rys. 4.35)
  2) Dodanie do aplikacji zainstalowanych w projekcie (rys. 4.36)
  3) Dodanie klasy pośredniczącej (middleware), która będzie nasłuchiwać odpowiedzi (rys. 4.37)
  4) Ustawienie listy źródeł upoważnionych do wykonywania żądań HTTP między witrynami w pliku ustawień (rys. 4.38)

```
django-cors-headers==4.3.1
```

Rysunek 4.34: Instalacja biblioteki django-cors-headers.

```
python -m pip install django-cors-headers
```

Rysunek 4.35: Instalacja biblioteki django-cors-headers.

```
INSTALLED_APPS = [
    ...,
    'corsheaders',
    ...,
]
```

Rysunek 4.36: Dodanie do aplikacji zainstalowanych w projekcie.

```
MIDDLEWARE = [
    ...,
    'corsheaders.middleware.CorsMiddleware',
    ...,
]
```

Rysunek 4.37: Dodanie klasy pośredniczącej.

```
CORS_ALLOWED_ORIGINS = [
    'http://localhost:5200',
]
```

Rysunek 4.38: Ustawienie listy dozwolonych domen do wykonania żądań CORS.

- **Testy jednostkowe i integracyjne**

  Ważnym aspektem zapewnienia jakości kodu są testy jednostkowe i integracyjne, które pozwalają na wczesne wykrywanie błędów zarówno w backendzie, jak i frontendzie. Dzięki tym testom możemy szybko identyfikować i naprawiać problemy, zanim trafią one do produkcji.

  Testy jednostkowe (unit tests) mają na celu weryfikację poprawności poszczególnych komponentów aplikacji, takich jak funkcje, klasy czy metody. Każdy test jednostkowy powinien być niezależny od innych i sprawdzać pojedynczy element kodu, aby upewnić się, że działa on zgodnie z oczekiwaniami. [21]

  W przypadku naszej aplikacji do nauki słówek, testy jednostkowe mogą obejmować:

  - **Funkcje związane z obsługą słówek:** Testowanie poprawności funkcji dodawania, usuwania i edytowania słówek.
  - **Autoryzacja i uwierzytelnianie użytkowników:** Sprawdzenie, czy metody set_password i check_password działają poprawnie.
  - **Walidacja danych:** Testowanie funkcji walidujących dane wejściowe użytkowników na poziomie backendu i frontendu.
  - **Logika biznesowa:** Weryfikacja poprawności algorytmów używanych w aplikacji, np. algorytmów oceniających postępy użytkownika.

  Przykładowy test jednostkowy w Django przedstawiono na rys. 4.39.

```python
from django.test import TestCase
from django.contrib.auth.models import User

class UserModelTest(TestCase):
    def test_create_user(self):
        user = User.objects.create_user(username='testuser', password='password')
        self.assertIsNotNone(user.id)
        self.assertTrue(user.check_password('password'))
```

Rysunek 4.39: Test jednostkowy.

Testy integracyjne (integration tests) sprawdzają, jak różne komponenty aplikacji współdziałają ze sobą. Celem jest wykrycie błędów wynikających z interakcji między poszczególnymi modułami systemu. [21]

W naszej aplikacji do nauki słówek testy integracyjne mogą obejmować:

- **Proces rejestracji i logowania użytkownika:** Sprawdzenie, czy użytkownik może poprawnie się zarejestrować, zalogować i uzyskać dostęp do chronionych zasobów.
- **Komunikacja między frontendem a backendem:** Testowanie, czy interfejs użytkownika prawidłowo komunikuje się z serwerem i poprawnie obsługuje odpowiedzi API.
- **Przepływ danych:** Weryfikacja, czy dane są poprawnie przesyłane, zapisywane i odczytywane z bazy danych podczas różnych operacji użytkownika.

Przykładowy test integracyjny w Django przedstawiono na rys. 4.40.

```python
from rest_framework.test import APITestCase
from django.urls import reverse
from rest_framework import status
from django.contrib.auth.models import User

class UserRegistrationTest(APITestCase):
    def test_register_user(self):
        url = reverse('register')
        data = {
            'username': 'newuser',
            'password': 'newpassword',
            'email': 'newuser@example.com'
        }
        response = self.client.post(url, data, format='json')
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)
        self.assertEqual(User.objects.count(), 1)
        self.assertEqual(User.objects.get().username, 'newuser')
```

Rysunek 4.40: Test integracyjny.

5. Faza wdrażania aplikacji - testy aplikacji

5.1. Proces wdrażania aplikacji

Git bash

GitBash to narzędzie oparte na Git, które umożliwia efektywne zarządzanie kodem źródłowym aplikacji. Dzięki niemu można śledzić zmiany w kodzie, tworzyć gałęzie (branches), łączyć zmiany (merge), a także cofać błędy (revert). Zapewnia on interfejs wiersza poleceń do pracy z systemem Git, co jest kluczowe dla współpracy zespołowej i śledzenia historii zmian w projekcie. Dzięki interfejsowi wiersza poleceń użytkownicy mają pełną kontrolę nad operacjami Git, co może być szczególnie przydatne w zaawansowanych scenariuszach pracy z repozytoriami. [19]

PyCharm Community

PyCharm Community to zaawansowane środowisko programistyczne (IDE), które wspiera pełen cykl życia projektu. Umożliwia tworzenie, edycję i debugowanie kodu, co jest niezbędne przy rozwijaniu aplikacji webowej. Dodatkowo oferuje zintegrowane wsparcie dla systemu kontroli wersji Git. Umożliwia zarządzanie gałęziami, przeglądanie historii zmian, rozwiązywanie konfliktów podczas scalania oraz wiele innych funkcji, które ułatwiają pracę zespołową nad projektem. Ponad to, oferuje także narzędzia do testowania, analizy kodu oraz debugowania, co jest istotne dla zapewnienia bezpieczeństwa aplikacji poprzez eliminację błędów i niepożądanych zachowań. [20]

Node.js

Node.js jest środowiskiem uruchomieniowym JavaScript, które umożliwia uruchamianie aplikacji JavaScript poza przeglądarką. Jest niezbędne do uruchamiania narzędzi, takich jak npm (Node Package Manager) i yarn, oraz do budowy aplikacji frontendowych w React.

Aby skonfigurować środowisko frontendowe dla naszego projektu aplikacji webowej do nauki słówek, koncentrując się na technologii React, należy zainstalować Node.js oraz odpowiednie narzędzia zarządzania pakietami JavaScript.

Uruchamianie aplikacji lokalnie

Aby uruchomić aplikację webową do nauki słówek lokalnie, należy uruchomić zarówno backend napisany w Django, jak i frontend oparty na React. Po uruchomieniu obu części aplikacji, należy przetestować interakcję między frontendem a backendem oraz upewnić się, że formularze, przepływ danych i funkcjonalności aplikacji działają zgodnie z oczekiwaniami.

Konfiguracja portów serwera aplikacyjnego:

1) Zmiana portu nasłuchiwania dla bazy danych w pliku konfiguracyjnym Postgresql (rys. 5.41)

```
postgresql.conf
54 #------------------------------------------------------------------------------
55 # CONNECTIONS AND AUTHENTICATION
56 #------------------------------------------------------------------------------
57
58 # - Connection Settings -
59
60 listen_addresses = '*'      # what IP address(es) to listen on;
61                             # comma-separated list of addresses;
62                             # defaults to 'localhost'; use '*' for all
63                             # (change requires restart)
64 port = 5335                 # (change requires restart)
65 max_connections = 100            # (change requires restart)
66 #reserved_connections = 0        # (change requires restart)
67 #superuser_reserved_connections = 3 # (change requires restart)
```

Rysunek 5.41: Zmiana portu nasłuchiwania.

2) Zmiana portu serwera aplikacji Django w skrypcie uruchamiającym (rys. 5.42)

```
runserver.py
1 import os
2 from django.core.management import execute_from_command_line
3
4 os.environ.setdefault("DJANGO_SETTINGS_MODULE", "vocabulary_app.settings")
5
6 execute_from_command_line(["manage.py", "runserver", "9000"])
```

Rysunek 5.42: Zmiana portu serwera - backend.

3) Zmiana portu serwera aplikacji React w ustawieniach konfiguracyjnych aplikacji (rys. 5.43)

```
JS vite.config.js M ×
JS vite.config.js > ...
1  import { defineConfig } from 'vite'
2  import react from '@vitejs/plugin-react'
3
4  // https://vitejs.dev/config/
5  export default defineConfig({
6    plugins: [react()],
7    server: {
8      port: 5200,
9    },
10 })
```

Rysunek 5.43: Zmiana portu serwera - frontend.

5.2. Testy aplikacji

Testy aplikacji przed jej wdrożeniem w środowisku produkcyjnym obejmują kilka rodzajów testów, które mają na celu upewnienie się, że aplikacja działa poprawnie i jest wolna od błędów.

Testy jednostkowe

Testy jednostkowe zostały szczegółowo omówione w rozdziale 4.4. "Kontrola kodu i zapewnienie bezpieczeństwa". Przed wdrożeniem aplikacji, wszystkie testy jednostkowe są uruchamiane, aby upewnić się, że poszczególne komponenty aplikacji działają zgodnie z oczekiwaniami.

Testy integracyjne

Testy integracyjne, również omówione w rozdziale 4.4., sprawdzają, czy różne moduły aplikacji współpracują ze sobą poprawnie. Przed wdrożeniem, uruchamiane są wszystkie testy integracyjne, aby wykryć potencjalne problemy wynikające z interakcji między komponentami.

Testy funkcjonalne

Testy funkcjonalne mają na celu weryfikację, czy aplikacja spełnia wszystkie wymagania funkcjonalne. W przypadku naszej aplikacji do nauki słówek, testy funkcjonalne obejmowałyby:

- Rejestrację nowego użytkownika

- Logowanie i wylogowywanie użytkownika
- Dodawanie, edytowanie i usuwanie słówek
- Przeglądanie listy słówek.

Przykładowy test funkcjonalny w Django przedstawiono na rys. 5.44.

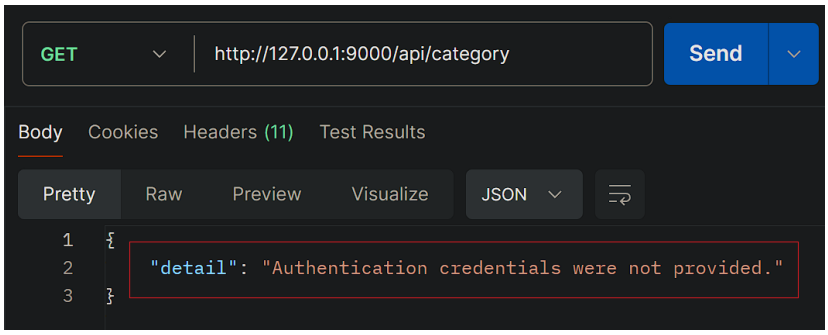
```python
from django.test import TestCase
from django.urls import reverse
from rest_framework import status
from rest_framework.test import APIClient
from django.contrib.auth.models import User

class FunctionalTestCase(TestCase):
    def setUp(self):
        self.client = APIClient()
        self.user = User.objects.create_user(username='testuser', password='testpassword')
        self.client.login(username='testuser', password='testpassword')

    def test_add_word(self):
        url = reverse('add_word')
        data = {'word': 'hello', 'translation': 'cześć'}
        response = self.client.post(url, data, format='json')
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)
```

Rysunek 5.44: Test funkcjonalny.

Testy bezpieczeństwa

Testy bezpieczeństwa mają na celu wykrycie potencjalnych luk i podatności w aplikacji. Przykładowy test bezpieczeństwa JWT przeprowadzony na naszej aplikacji do nauki słówek:

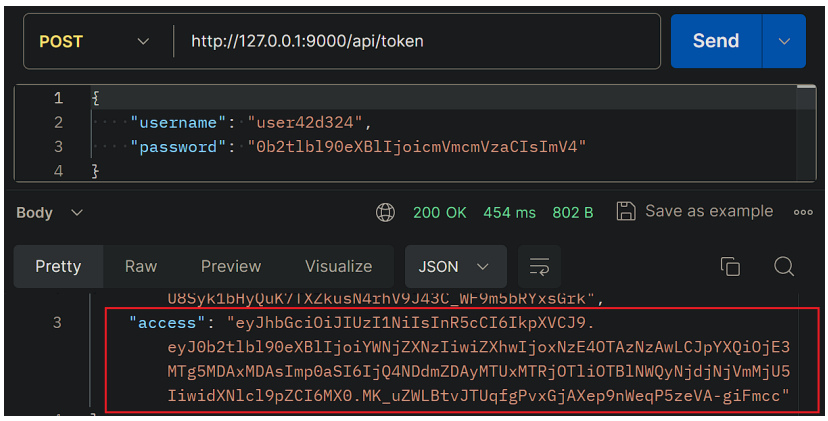
1) Próba nieautoryzowanego dostępu do API (rys. 5.45)



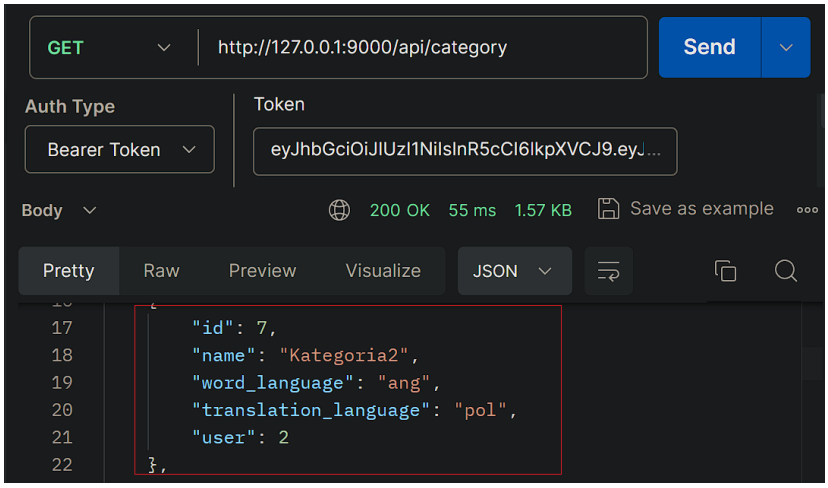
Rysunek 5.45: Test bezpieczeństwa - krok 1.

2) Uzyskanie tokenu (rys. 5.46)

3) Próba dostępu do API przy użyciu uzyskanego tokena (rys. 5.47)



Rysunek 5.46: Test bezpieczeństwa - krok 2.



Rysunek 5.47: Test bezpieczeństwa - krok 3.

Do przeprowadzenia testów bezpieczeństwa warto wykorzystać narzędzia takie jak Burp Suite i OWASP ZAP.

Burp Suite to zaawansowane narzędzie do testowania bezpieczeństwa aplikacji webowych, szeroko wykorzystywane przez pentesterów i specjalistów ds. bezpieczeństwa, które umożliwia przeprowadzanie kompleksowych testów, takich jak:

- Przechwytywanie i modyfikowanie żądań HTTP
- Wykrywanie luk w zabezpieczeniach
- Testy penetracyjne aplikacji webowych.

Przykład użycia Burp Suite:

1) Skonfigurowanie Burp Suite jako proxy w przeglądarce.

2) Przechwycenie żądania HTTP między przeglądarką a serwerem aplikacji.

3) Analiza i modyfikacja żądania w celu wykrycia podatności. [22]

OWASP ZAP (Zed Attack Proxy) to narzędzie open-source do testowania bezpieczeństwa aplikacji webowych, rozwijane i utrzymywane przez OWASP (Open Web Application Security Project). Jest darmowe i open-source, co czyni je dostępnym dla szerokiego grona użytkowników, od początkujących po doświadczonych pentesterów. OWASP ZAP oferuje:

- Automatyczne skanowanie aplikacji pod kątem luk bezpieczeństwa
- Możliwość ręcznego przeprowadzania testów penetracyjnych
- Analizowanie i modyfikowanie żądań HTTP.

Przykład użycia OWASP ZAP:

1) Uruchomienie OWASP ZAP i skonfigurowanie przeglądarki, aby korzystała z proxy ZAP.

2) Przeglądanie aplikacji, aby OWASP ZAP mógł przechwytywać żądania.

3) Wykonanie automatycznego skanowania aplikacji w poszukiwaniu podatności.

4) Przeanalizowanie wyników skanowania i naprawienie wykrytych problemów. [23]

6. Wnioski

Podczas realizacji projektu aplikacji webowej do nauki słówek, skupiliśmy się na wielu aspektach bezpieczeństwa, aby zapewnić użytkownikom bezpieczne i niezawodne narzędzie. Proces ten obejmował planowanie, projektowanie, implementację oraz testowanie aplikacji.

Ważnym aspektem jest odpowiednie wybranie technologii przy tworzeniu aplikacji. Django (jako backend) zapewnia solidne mechanizmy bezpieczeństwa, takie jak wbudowane funkcje uwierzytelniania oparte na JWT oraz zintegrowane zabezpieczenia przeciwko wielu rodzajom ataków. React (jako frontend) z kolei umożliwia efektywne zarządzanie interfejsem użytkownika, co przyczynia się do lepszej użyteczności i bezpieczeństwa aplikacji. Zastosowaliśmy walidację danych na poziomie frontendu, backendu oraz bazy danych, co znacząco zwiększyło bezpieczeństwo aplikacji i integralność danych. Z powodu ograniczeń czasowych nie udało się wdrożyć HTTPS do szyfrowania komunikacji pomiędzy klientem a serwerem, aczkolwiek jest to istotny aspekt do zapewnienia poufności przesyłanych danych. Implementacja JWT jako mechanizmu uwierzytelniania zapewniła bezpieczne logowanie użytkowników oraz dostęp do chronionych zasobów. Pomimo zastosowania int jako identyfikatorów w naszej bazie danych, zaleca się użycie GUID dla większego bezpieczeństwa. Warto w przyszłości wdrożyć dodatkowe mechanizmy, takie jak blokada czasowa po kilku nieudanych próbach logowania, wymuszanie zmiany hasła co pewien czas oraz dwuskładnikowe uwierzytelnianie (2FA). Zastosowanie pliku .env do przechowywania wrażliwych danych konfiguracyjnych oraz dodanie go do .gitignore zapewni, że te dane nie zostaną przypadkowo udostępnione publicznie. Podkreśliliśmy znaczenie testów jednostkowych i integracyjnych, które pozwalają na wczesne wykrywanie błędów zarówno na poziomie backendu, jak i frontendu. W przyszłości warto skupić się na implementacji bardziej rozbudowanych testów bezpieczeństwa, aby zapewnić jeszcze większą niezawodność aplikacji. Warto także rozważyć wdrożenie dodatkowych zabezpieczeń, takich jak pełne szyfrowanie danych użytkowników oraz regularne audyty bezpieczeństwa. Należy na bieżąco monitorować rozwój technologii i implementować nowe standardy w celu ciągłego podnoszenia poziomu bezpieczeństwa aplikacji.

7. Literatura

Literatura

[1] https://docs.djangoproject.com/en/5.0/ (dostęp: 31.04.2024r.)

[2] https://flask-cn.readthedocs.io/_/downloads/en/stable/pdf/ (dostęp: 17.05.2024r.)

[3] https://expressjs.com (dostęp: 17.05.2024r.)

[4] https://gorails.com (dostęp: 17.05.2024r.)

[5] https://github.com/facebook/react/blob/main/CHANGELOG.md#1562 (dostęp: 25.05.2024r.)

[6] https://merce.com/blog/react-angular-vue-krotkie-porownanie (dostęp: 25.05.2024r.)

[7] https://smartbees.pl/baza-wiedzy/svelte (dostęp: 25.05.2024r.)

[8] https://www.mongodb.com/resources/compare/mongodb-postgresql (dostęp: 08.06.2024r.)

[9] https://docs.djangoproject.com/en/5.0/releases/ (dostęp: 08.06.2024r.)

[10] https://docs.djangoproject.com/en/5.0/topics/security/ (dostęp: 08.06.2024r.)

[11] https://www.postgresql.org/docs/current/ (dostęp: 08.06.2024r.)

[12] https://boringowl.io/blog/jak-poprawnie-przeprowadzic-walidacje-danych (dostęp: 15.06.2024r.)

[13] https://docs.djangoproject.com/pl/5.0/topics/forms/modelforms/ (dostęp: 15.06.2024r.)

[14] https://www.postgresql.org/docs/current/sql-createtable.html (dostęp: 15.06.2024r.)

[15] https://www.cloudflare.com/pl-pl/learning/ssl/what-is-https/ (dostęp: 15.06.2024r.)

[16] https://devszczepaniak.pl/json-web-token/ (dostęp: 15.06.2024r.)

[17] https://www.digitalguardian.com/blog/what-principle-least-privilege-polp-best-practice-information-security-and-compliance (dostęp: 15.06.2024r.)

[18] https://sekurak.pl/czym-jest-cors-cross-origin-resource-sharing-i-jak-wplywa-na-bezpieczenstwo/ (dostęp: 15.06.2024r.)

[19] https://www.atlassian.com/git/tutorials/git-bash (dostęp: 22.06.2024r.)

[20] https://www.jetbrains.com/help/pycharm/installation-guide.html (dostęp: 22.06.2024r.)

[21] https://devenv.pl/poziomy-testow/ (dostęp: 22.06.2024r.)

[22] https://sekurak.pl/mastering-burp-suite-cz-1-poznaj-sekrety-topowego-narzedzia-dla-pentesterow-web-aplikacji/ (dostęp: 22.06.2024r.)

[23] https://sekurak.pl/wprowadzenie-do-narzedzia-zed-attack-proxy-zap/ (dostęp: 22.06.2024r.)